Article

Creating a word tagger model

Train a machine learning model to tag individual words in natural language text.

Overview

A *word tagger* is a machine learning model that's been trained to classify natural language text at the word level.

You train a word tagger by showing it multiple examples of sentences containing words you've already tagged — for example, Apple product names like *iPad* and *iPhone*.

Import your data

Start by gathering textual data and importing it into an `MLDataTable` instance. You can create a data table from JSON and CSV formats.

As an example, consider a JSON file containing sentences with words that you've tagged. Each entry contains a pair of keys—`tokens` and `labels`:

- The value for the `tokens` key is an array of words and punctuation in an individual sentence.
- The value for `labels` is an array of corresponding labels, or tags, for each of those tokens.

The arrays are the same length, resulting in a one-to-one mapping between each token and its corresponding label.

The JSON snippet below shows three pairs of tokenized sentences with their associated labels.

```
// JSON file
[
    {
        "tokens": ["AirPods", "are", "a", "fantastic", "Apple", "product", "."],
        "labels": ["PROD", "NONE", "NONE", "NONE", "ORG", "NONE", "NONE"]
    },
    {
        "tokens": ["The", "iPhone", "takes", "stunning", "photos", "."],
        "labels": ["NONE", "PROD", "NONE", "NONE", "NONE", "NONE"]
    },
    {
        "tokens": ["Start", "building", "a", "native", "Mac", "app", "from", "your
        "labels": ["NONE", "NONE", "NONE", "NONE", "PROD", "NONE", "NONE", "NONE",
    }
]
```

In a macOS playground, create the data table using the `init(contentsOf:options:)` method of `MLDataTable`.

```
import CreateML

let data = try MLDataTable(contentsOf:
    URL(fileURLWithPath: "<#/path/to/read/data.json#>"))
```

The resulting data table has two columns, named `tokens` and `labels`, derived from the keys in the JSON file. The column names can be anything, as long as they're meaningful to you, because you'll use them as parameters in other methods.

You'll use the data you gathered for two critical tasks: model training and evaluation.

Prepare your data for training and evaluation

After you train your word tagger model, you need to evaluate how good it's at its tagging task. To test the model's performance, set aside part of your data in a testing dataset. Training and testing data must be entirely separate, with no overlap. This way, the metrics you calculate based on your testing data tell you how well the classifier performs on examples it hasn't already seen.

Generally, the testing dataset is significantly smaller than the training dataset. You might want to use about 80 percent of your overall data to train the model, and 20 percent to test it. However, it's more important that the testing data is high-quality: as close as possible to real-world examples, well-distributed, and balanced. Use your best data for your testing data.

One way to generate training and testing data from your dataset is to use the `randomSplit(by:seed:)` method of `MLDataTable`. This method splits your data into two tables, one for training and the other for testing. Specify a `0.8` split to create a training data table that contains 80 percent of your data, and a testing data table that contains the remaining 20 percent.

```
let (trainingData, testingData) = data.randomSplit(by: 0.8, seed: 5)
```

Although `randomSplit(by:seed:)` provides a quick way to split your dataset, consider creating separate training and testing datasets manually. That way, you can ensure that your testing data is high quality.

Create and train the word tagger

Create an instance of `MLWordTagger` with your training data table and the names of your columns. Training begins immediately.

```
let wordTagger = try MLWordTagger(trainingData: trainingData,
                                  tokenColumn: "tokens",
                                  labelColumn: "labels")
```

Word tagger training requires setting aside a small subset of the training data as a validation dataset, to help keep track of training progress. The validation data allows the training process to gauge the model's performance on examples the model hasn't been trained on. Depending on the validation accuracy, the training algorithm could adjust values within the model or even stop the training process, if the accuracy is high enough.

Training and validation data must be entirely separate, with no overlap. If you want to make this split yourself, you can set aside around 5 to 10 percent of your training data as your validation data, and provide that data by setting custom model parameters. If you don't do the split yourself, Create ML automatically sets aside a small percentage of the training data to use for

validating the model's progress during the training phase. Because the data is split randomly, you might get a different result each time you train the model.

To see how accurately the model performed on the training and validation data, use the `taggingError` property of the model's `trainingMetrics` and `validationMetrics` properties.

```
// Training accuracy as a percentage
let trainingAccuracy = (1.0 - wordTagger.trainingMetrics.taggingError) * 100

// Validation accuracy as a percentage
let validationAccuracy = (1.0 - wordTagger.validationMetrics.taggingError) * 100
```

Evaluate the tagger's accuracy

Next, evaluate your trained model's performance by testing it against sentences it's never seen before. Pass your testing data table to the `evaluation(on:tokenColumn:labelColumn:)` method, which returns an `MLWordTaggerMetrics` instance.

```
let evaluationMetrics = wordTagger.evaluation(on: testingData,
                                              tokenColumn: "tokens",
                                              labelColumn: "labels")
```

To get the evaluation accuracy, use the `taggingError` property of the returned `MLWordTaggerMetrics` instance.

```
// Evaluation accuracy as a percentage
let evaluationAccuracy = (1.0 - evaluationMetrics.taggingError) * 100
```

If the evaluation performance isn't good enough, you may need to retrain with more data or make other adjustments. For information about improving model performance, see `Improving Your Model's Accuracy`.

Save the Core ML model

When your model is performing well enough, you're ready to save it so you can use it in your app. Use the `write(to:metadata:)` method to write the Core ML model file (in this example, `AppleTagger.mlmodel`) to disk. Provide any information about the model, like its author, version, or description, in an `MLModelMetadata` instance.

```
let metadata = MLModelMetadata(author: "Jane Appleseed",
                               shortDescription: "A model trained to tag Apple pro
                               version: "1.0")

try wordTagger.write(to: URL(fileURLWithPath: "<#/path/to/save/AppleTagger.mlmodel
                             metadata: metadata)
```

Add the model to your app

With your app open in Xcode, drag the `AppleTagger.mlmodel` file into the navigation pane. Xcode compiles the model and generates an `AppleTagger` class for use in your app. Select the `AppleTagger.mlmodel` file in Xcode to view additional information about the model.

Create an `NLModel` in the Natural Language framework from the `AppleTagger` to ensure that the tokenization is consistent between training and deployment. Attach the model to an `NLTagger` to tag sentences or paragraphs, using an existing or custom `NLTagScheme`.

```
import NaturalLanguage
import CoreML

let text = "The iPad is my favorite Apple product."

do {
    let mlModel = try AppleTagger(configuration: MLModelConfiguration()).model

    let customModel = try NLModel(mlModel: mlModel)
    let customTagScheme = NLTagScheme("Apple")

    let tagger = NLTagger(tagSchemes: [.nameType, customTagScheme])
    tagger.string = text
    tagger.setModels([customModel], forTagScheme: customTagScheme)
```

```
    tagger.enumerateTags(in: text.startIndex..<text.endIndex, unit: .word,
                         scheme: customTagScheme, options: .omitWhitespace) { tag,
        if let tag = tag {
            print("\(text[tokenRange]): \(tag.rawValue)")
        }
        return true
    }
} catch {
    print(error)
```

See Also

Natural language models

Creating a text classifier model

Train a machine learning model to classify natural language text.

`class NLModel`

A custom model trained to classify or tag natural language text.